Week6: SUPERMARKET MANAGEMENT SYSTEM

Product Review and Rating Management

Name: Logeshwari D
Register Number: E25AI121

---

1. Scenario

The Supermarket Management System is extended with a Product Review and Rating Module to collect customer feedback about purchased products.

Customers can provide ratings and reviews for products. The system stores customer opinions, analyzes product performance, and identifies highly rated products and products requiring improvement.

The module is connected with the existing CUSTOMER and PRODUCT tables. It helps the supermarket understand customer satisfaction and improve product quality.

2. Create Review and Rating Table

The PRODUCT_REVIEW table stores customer reviews and ratings for supermarket products.

```
SQL>
SQL> CREATE TABLE PRODUCT_REVIEW (
  2      REVIEW_ID NUMBER PRIMARY KEY,
  3      CUSTOMER_ID NUMBER NOT NULL,
  4      PRODUCT_ID VARCHAR2(10) NOT NULL,
  5      RATING NUMBER(1) NOT NULL,
  6      REVIEW_COMMENT VARCHAR2(500),
  7      REVIEW_DATE DATE DEFAULT SYSDATE,
  8      CONSTRAINT FK_REVIEW_CUSTOMER
  9          FOREIGN KEY (CUSTOMER_ID)
 10          REFERENCES CUSTOMER(ID),
 11      CONSTRAINT FK_REVIEW_PRODUCT
 12          FOREIGN KEY (PRODUCT_ID)
 13          REFERENCES PRODUCT(PRODUCT_ID),
 14      CONSTRAINT CHK_REVIEW_RATING
 15          CHECK (RATING BETWEEN 1 AND 5)
 16  );

Table created.
```

Verify the Table

The DESC command displays the structure of the PRODUCT_REVIEW table.

```
SQL> DESC PRODUCT_REVIEW;
 Name                                      Null?    Type
 ----------------------------------------- -------- ------------
 REVIEW_ID                                 NOT NULL NUMBER
 CUSTOMER_ID                               NOT NULL NUMBER
 PRODUCT_ID                                NOT NULL VARCHAR2(10)
 RATING                                    NOT NULL NUMBER(1)
 REVIEW_COMMENT                                     VARCHAR2(500)
 REVIEW_DATE                                        DATE
```

3. Store Customer Feedback and Ratings

Task

1. Add reviews for purchased products

```
SQL> SELECT * FROM PRODUCT_REVIEW;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------
         1           1 P001                5
Excellent product
01-NOV-26

         2           2 P002                4
Good quality
01-NOV-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------

         3           3 P003                3
Average product
01-NOV-26

         4           4 P004                5
Very refreshing juice

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------
01-NOV-26

         5           5 P005                2
Not very satisfied
01-NOV-26

         6           1 P001                4

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------
Good product
01-NOV-26


6 rows selected.
```

2. Stores customers ratings from 1 to 5 stars

   Ensure this by adding constraint

```
SQL> ALTER TABLE PRODUCT_REVIEW
  2  ADD CONSTRAINT CHECK_RATING_RANGE
  3  CHECK (RATING >= 1 AND RATING <= 5);

Table altered.
```

```
SQL> INSERT INTO PRODUCT_REVIEW
  2  (REVIEW_ID, CUSTOMER_ID, PRODUCT_ID, RATING, REVIEW_COMMENT)
  3  VALUES (7, 1, 'P001', 6, 'Test rating');
INSERT INTO PRODUCT_REVIEW
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CHK_REVIEW_RATING) violated
```

3. Update review command if required

```
SQL> UPDATE PRODUCT_REVIEW
  2  SET REVIEW_COMMENT = 'Good quality and value'
  3  WHERE REVIEW_ID = 2;

1 row updated.
```

4. Remove inappropriate or invalid reviews

```
SQL> DELETE FROM PRODUCT_REVIEW
  2  WHERE RATING NOT BETWEEN 1 AND 5
  3     OR REVIEW_COMMENT IS NULL;

0 rows deleted.
```

4.Retrieve product review details

1.Display all reviews for a product

```
SQL> SELECT REVIEW_ID,
  2         CUSTOMER_ID,
  3         PRODUCT_ID,
  4         RATING,
  5         REVIEW_COMMENT,
  6         REVIEW_DATE
  7  FROM PRODUCT_REVIEW
  8  WHERE PRODUCT_ID = 'P001';

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
         1           1 P001                5
Excellent product
01-NOV-26

         6           1 P001                4
Good product
01-NOV-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------

         7           1 P001                5
Excellent product
01-NOV-26
```

2.Display Customer Names with Their Reviews

```
SQL> SELECT C.NAME,
  2         PR.PRODUCT_ID,
  3         PR.RATING,
  4         PR.REVIEW_COMMENT,
  5         PR.REVIEW_DATE
  6  FROM CUSTOMER C
  7  JOIN PRODUCT_REVIEW PR
  8  ON C.ID = PR.CUSTOMER_ID
  9  ORDER BY C.NAME;

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------
REVIEW_DA
---------
Chaitali                       P004                5
Very refreshing juice
01-NOV-26

Hardik                         P005                2
Not very satisfied
01-NOV-26

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------
REVIEW_DA
---------

Khilan                         P002                4
Good quality and value
01-NOV-26

Ramesh                         P001                5
Excellent product

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------
REVIEW_DA
---------
01-NOV-26

Ramesh                         P001                4
Good product
01-NOV-26

Ramesh                         P001                5

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------
REVIEW_DA
---------
Excellent product
01-NOV-26

kaushik                        P003                3
Average product
01-NOV-26


7 rows selected.
```

3.Find Product Having Maximum Reviews

```
SQL> SELECT PRODUCT_ID,
  2         COUNT(*) AS TOTAL_REVIEWS
  3  FROM PRODUCT_REVIEW
  4  GROUP BY PRODUCT_ID
  5  HAVING COUNT(*) = (
  6      SELECT MAX(REVIEW_COUNT)
  7      FROM (
  8          SELECT COUNT(*) AS REVIEW_COUNT
  9          FROM PRODUCT_REVIEW
 10          GROUP BY PRODUCT_ID
 11      )
 12  );

PRODUCT_ID TOTAL_REVIEWS
---------- -------------
P001                 3
```

4.Display Recent Customer Feedback

```
SQL> SELECT C.NAME,
  2         PR.PRODUCT_ID,
  3         PR.RATING,
  4         PR.REVIEW_COMMENT,
  5         PR.REVIEW_DATE
  6  FROM CUSTOMER C
  7  JOIN PRODUCT_REVIEW PR
  8  ON C.ID = PR.CUSTOMER_ID
  9  ORDER BY PR.REVIEW_DATE DESC;

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Ramesh                         P001                5
Excellent product
01-NOV-26

Chaitali                       P004                5
Very refreshing juice
01-NOV-26

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------

Ramesh                         P001                4
Good product
01-NOV-26

Hardik                         P005                2
Not very satisfied

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
01-NOV-26

Ramesh                         P001                5
Excellent product
01-NOV-26

kaushik                        P003                3

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Average product
01-NOV-26

Khilan                         P002                4
Good quality and value
01-NOV-26


7 rows selected.
```

5.Retrieve Reviews with Ratings Above 4

```
SQL> SELECT C.NAME,
  2         PR.PRODUCT_ID,
  3         PR.RATING,
  4         PR.REVIEW_COMMENT,
  5         PR.REVIEW_DATE
  6  FROM CUSTOMER C
  7  JOIN PRODUCT_REVIEW PR
  8  ON C.ID = PR.CUSTOMER_ID
  9  WHERE PR.RATING > 4
 10  ORDER BY PR.RATING DESC;

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------
REVIEW_DA
---------
Ramesh                         P001                5
Excellent product
01-NOV-26

Chaitali                       P004                5
Very refreshing juice
01-NOV-26

NAME                           PRODUCT_ID     RATING
------------------------------ ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------
REVIEW_DA
---------

Ramesh                         P001                5
Excellent product
01-NOV-26
```

5.Calculate Average Product Ratings

Use aggregate functions to analyze ratings

Tasks:

1.calculate average rating for each product

```
SQL> SELECT C.NAME,
  2         AVG(PR.RATING) AS AVERAGE_RATING,
  3         COUNT(PR.REVIEW_ID) AS TOTAL_REVIEWS
  4  FROM CUSTOMER C
  5  JOIN PRODUCT_REVIEW PR
  6  ON C.ID = PR.CUSTOMER_ID
  7  GROUP BY C.NAME
  8  ORDER BY AVERAGE_RATING DESC;

NAME                           AVERAGE_RATING TOTAL_REVIEWS
------------------------------ -------------- -------------
Chaitali                                    5             1
Ramesh                             4.66666667             3
Khilan                                      4             1
kaushik                                     3             1
Hardik                                      2             1
```

*2.*Count the Number of Reviews per Customer

```
SQL> SELECT C.NAME,
  2         COUNT(PR.REVIEW_ID) AS TOTAL_REVIEWS
  3  FROM CUSTOMER C
  4  JOIN PRODUCT_REVIEW PR
  5  ON C.ID = PR.CUSTOMER_ID
  6  GROUP BY C.NAME
  7  ORDER BY TOTAL_REVIEWS DESC;

NAME                           TOTAL_REVIEWS
------------------------------ -------------
Ramesh                                     3
Khilan                                     1
Hardik                                     1
Chaitali                                   1
kaushik                                    1
```

3.Find the Highest-Rated Product

```
SQL> SELECT PRODUCT_ID,
  2         AVG(RATING) AS AVERAGE_RATING
  3  FROM PRODUCT_REVIEW
  4  GROUP BY PRODUCT_ID
  5  HAVING AVG(RATING) = (
  6      SELECT MAX(AVG_RATING)
  7      FROM (
  8          SELECT AVG(RATING) AS AVG_RATING
  9          FROM PRODUCT_REVIEW
 10          GROUP BY PRODUCT_ID
 11      )
 12  );

PRODUCT_ID AVERAGE_RATING
---------- --------------
P004                    5
```

4.Identify Products with Average Rating Above 4

```
SQL> SELECT PRODUCT_ID,
  2         AVG(RATING) AS AVERAGE_RATING
  3  FROM PRODUCT_REVIEW
  4  GROUP BY PRODUCT_ID
  5  HAVING AVG(RATING) > 4
  6  ORDER BY AVERAGE_RATING DESC;

PRODUCT_ID AVERAGE_RATING
---------- --------------
P004                    5
P001           4.66666667
```

5. Identify Highly Rated Products

Generate product rating reports.

Report 1:Product Rating

Analysis Display:

- Product name
- Number of reviews
- Average rating

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(PR.REVIEW_ID) AS NUMBER_OF_REVIEWS,
  3         ROUND(AVG(PR.RATING), 2) AS AVERAGE_RATING
  4  FROM PRODUCT P
  5  JOIN PRODUCT_REVIEW PR
  6  ON P.PRODUCT_ID = PR.PRODUCT_ID
  7  GROUP BY P.PRODUCT_NAME
  8  ORDER BY AVERAGE_RATING DESC;

PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Orange Juice
                1              5

Rice 5kg
                3           4.67

Wheat Flour 1kg
                1              4


PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Sugar 1kg
                1              3

Green Tea
                1              2
```

Report 2: Customer Feedback Analysis

Find:

● Most reviewed products

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(PR.REVIEW_ID) AS TOTAL_REVIEWS
  3  FROM PRODUCT P
  4  JOIN PRODUCT_REVIEW PR
  5  ON P.PRODUCT_ID = PR.PRODUCT_ID
  6  GROUP BY P.PRODUCT_NAME
  7  ORDER BY TOTAL_REVIEWS DESC;

PRODUCT_NAME
--------------------------------------------------
TOTAL_REVIEWS
-------------
Rice 5kg
            3

Sugar 1kg
            1

Green Tea
            1


PRODUCT_NAME
--------------------------------------------------
TOTAL_REVIEWS
-------------
Orange Juice
            1

Wheat Flour 1kg
            1
```

● Highly rated products

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(PR.REVIEW_ID) AS TOTAL_REVIEWS,
  3         ROUND(AVG(PR.RATING), 2) AS AVERAGE_RATING
  4  FROM PRODUCT P
  5  JOIN PRODUCT_REVIEW PR
  6  ON P.PRODUCT_ID = PR.PRODUCT_ID
  7  GROUP BY P.PRODUCT_NAME
  8  HAVING AVG(PR.RATING) >= 4
  9  ORDER BY AVERAGE_RATING DESC;

PRODUCT_NAME
--------------------------------------------------
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Orange Juice
            1              5

Rice 5kg
            3           4.67

Wheat Flour 1kg
            1              4
```

● Products requiring improvement

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(PR.REVIEW_ID) AS TOTAL_REVIEWS,
  3         ROUND(AVG(PR.RATING), 2) AS AVERAGE_RATING
  4  FROM PRODUCT P
  5  JOIN PRODUCT_REVIEW PR
  6  ON P.PRODUCT_ID = PR.PRODUCT_ID
  7  GROUP BY P.PRODUCT_NAME
  8  HAVING AVG(PR.RATING) < 3
  9  ORDER BY AVERAGE_RATING ASC;

PRODUCT_NAME
--------------------------------------------------------
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Green Tea
            1              2
```

Report 3: Rating Distribution

● Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
  2  FROM PRODUCT_REVIEW
  3  WHERE RATING = 5;

FIVE_STAR_RATINGS
-----------------
                3
```

● Number of 4-star ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
  2  FROM PRODUCT_REVIEW
  3  WHERE RATING = 4;

FOUR_STAR_RATINGS
-----------------
                2
```

● Number of low-rated products

```
SQL> SELECT COUNT(*) AS LOW_RATED_PRODUCTS
  2  FROM (
  3      SELECT PRODUCT_ID,
  4             AVG(RATING) AS AVERAGE_RATING
  5      FROM PRODUCT_REVIEW
  6      GROUP BY PRODUCT_ID
  7      HAVING AVG(RATING) < 3
  8  );

LOW_RATED_PRODUCTS
------------------
                 1
```

● Complete Rating

```
SQL> SELECT RATING,
  2         COUNT(*) AS TOTAL_REVIEWS
  3  FROM PRODUCT_REVIEW
  4  GROUP BY RATING
  5  ORDER BY RATING DESC;

    RATING TOTAL_REVIEWS
---------- -------------
         5             3
         4             2
         3             1
         2             1
```